

SC2001 Project 1

Team 7

Members:

MADHAV RAGHU ANANTHARAM

GONG YICHEN

SEAN TEO WEI XIANG

LYE CHEE HUNG IRWIN



Algorithm Overview

Key Idea:

- Merge Sort is efficient for large arrays, but incurs high recursion overhead for small subarrays. Hybridize by switching to Insertion Sort when subarray size $\leq S$ (threshold)
- Insertion Sort is faster for small subarrays, and doesn't require recursion

Algorithm Steps:

- If subarray size $> S$: Recursively sort left and right halves, then merge
- If subarray size $\leq S$: Use Insertion Sort



Pseudocode

```
hybridSort(arr, left, right, S){  
    if(left >= right) return;  
    size = right - left + 1;  
    if(size <= S){  
        insertionSort(arr, left, right);  
    }
```

```
else{  
    mid = left + (right - left) // 2;  
    hybridSort(arr, left, mid, S);  
    hybridSort(arr, mid+1, right, S);  
    merge(arr, left, mid, right);  
}
```



Recurrence Relations

Definition: Let $T(n)$ be the time complexity (or number of key comparisons in worst case) for sorting n elements.

Recurrence for Hybrid Sort:

- Base case:

$$\text{If } n \leq S, T(n) = \Theta(n^2)$$

(Insertion Sort: Worst-case comparisons $\approx n(n-1)/2$, from shifting elements.)

- Recursive case:

$$\text{If } n > S, T(n) = \Theta(n \log(n))$$

(Two recursive calls + merge cost; merge comparisons $\leq n-1$ in worst case.)

b) Generating the input data

```
size = int(input("Enter size of array: "))  
maxVal = int(input("Enter maximum value: "))
```

Enter size of array: 232

Enter maximum value: 22

```
[22, 22, 21, 16, 13, 8, 3, 12, 6, 0, 17, 17, 18, 5, 3, 13, 1, 15, 5, 3, 16, 20, 5, 0, 12, 18, 8, 4, 17, 18, 14, 19, 17, 14, 8, 18, 4, 8, 20, 1, 20, 17, 13, 22, 13, 15, 22, 12, 5, 5,  
20, 10, 21, 4, 12, 22, 15, 14, 16, 12, 3, 20, 11, 4, 16, 22, 15, 22, 0, 5, 2, 21, 19, 7, 14, 15, 15, 5, 0, 18, 16, 0, 19, 8, 20, 0, 4, 13, 19, 7, 15, 10, 10, 17, 3, 11, 19, 16, 6,  
15, 10, 3, 18, 21, 6, 4, 7, 9, 19, 18, 9, 12, 22, 13, 3, 22, 22, 15, 22, 7, 14, 5, 12, 10, 19, 1, 18, 10, 15, 3, 16, 8, 7, 11, 0, 4, 10, 16, 6, 12, 14, 18, 21, 15, 19, 7, 20, 2, 3,  
11, 9, 6, 15, 16, 17, 14, 3, 16, 16, 9, 17, 5, 12, 20, 17, 10, 4, 12, 8, 16, 20, 20, 5, 9, 20, 21, 1, 21, 20, 17, 1, 17, 6, 7, 19, 21, 21, 20, 6, 1, 8, 16, 4, 4, 3, 5, 12, 11, 19, 1  
3, 5, 4, 1, 7, 16, 1, 0, 4, 10, 10, 4, 10, 22, 19, 18, 20, 9, 17, 20, 22, 4, 10, 16, 18, 8, 5, 16, 14, 4, 14, 18, 4]
```

```
def generate_random_array(size, maxVal):  
    return [random.randint(1, maxVal) for i in range(size)]
```

c(i) Generate dataset for fixed S

```
n = [1000, 2000, 4000, 8000, 16000, 32000]
fixedS = 16
```

```
# function to test varying n to fixedS for part C (i)
def testloopfixedS(n, fixedS, max_val=10000000):
    global comparisons
    for size in n:
        arr = generate_random_array(size, max_val)
        comparisons = 0
        start = time.perf_counter()
        hybridSort(arr, 0, len(arr)-1, fixedS)
        end = time.perf_counter()
        timeTaken = end - start
        print(f"Fixed S={fixedS}, n={size}, # of comparisons={comparisons}, Time taken={timeTaken:.6f} seconds")
    return
```

c(ii) Generate dataset for fixed n

```
fixedn = 8000  
Svalues = [2, 4, 8, 16, 32, 64]
```

```
#function to test fixed n to varying S for part C(ii)  
def testloopfixedn(fixedn, Svalues, max_val = 10000000):  
    global comparisons  
    for S in Svalues:  
        arr = generate_random_array(fixedn, max_val)  
        comparisons = 0  
        start = time.perf_counter()  
        hybridSort(arr, 0, len(arr)-1, S)  
        end = time.perf_counter()  
        timeTaken = end - start  
        print(f"Fixed n={fixedn}, S={S}, # of comparisons={comparisons}, Time taken={timeTaken:.6f} seconds")  
    return
```


c(iii)

(C) Part (i): Comparisons over different n with fixed S
 Fixed S=16, n=1000, # of comparisons=8937, Time taken=0.001559 seconds
 Fixed S=16, n=2000, # of comparisons=19448, Time taken=0.002016 seconds
 Fixed S=16, n=4000, # of comparisons=42405, Time taken=0.004904 seconds
 Fixed S=16, n=8000, # of comparisons=91993, Time taken=0.010593 seconds
 Fixed S=16, n=16000, # of comparisons=197939, Time taken=0.023520 seconds
 Fixed S=16, n=32000, # of comparisons=423228, Time taken=0.050935 seconds

opt
opt

(C) Part (ii): Comparisons over different s with fixed n
 Fixed n=8000, S=2, # of comparisons=85363, Time taken=0.011704 seconds
 Fixed n=8000, S=4, # of comparisons=85078, Time taken=0.010821 seconds
 Fixed n=8000, S=8, # of comparisons=85984, Time taken=0.011502 seconds
 Fixed n=8000, S=16, # of comparisons=91578, Time taken=0.010639 seconds
 Fixed n=8000, S=32, # of comparisons=108514, Time taken=0.010789 seconds
 Fixed n=8000, S=64, # of comparisons=148092, Time taken=0.012920 seconds

def testlooptoptimalS(

global comparison

fixed_arrays = {n

for n in optn:

for S in optS

arr = fix

compariso

start = t

hybridSor

end = tim

timeTaken

print(f"

return

(C) Part (iii): Test for Optimal S

Optimal S=2, n=1000, # of comparisons=8002, Time taken=0.001122 seconds
 Optimal S=16, n=1000, # of comparisons=8831, Time taken=0.000843 seconds
 Optimal S=64, n=1000, # of comparisons=16300, Time taken=0.001313 seconds
 Optimal S=128, n=1000, # of comparisons=27323, Time taken=0.001943 seconds
 Optimal S=2, n=8000, # of comparisons=85328, Time taken=0.012468 seconds
 Optimal S=16, n=8000, # of comparisons=92386, Time taken=0.010287 seconds
 Optimal S=64, n=8000, # of comparisons=150867, Time taken=0.014069 seconds
 Optimal S=128, n=8000, # of comparisons=237897, Time taken=0.019769 seconds
 Optimal S=2, n=32000, # of comparisons=397716, Time taken=0.056940 seconds
 Optimal S=16, n=32000, # of comparisons=425109, Time taken=0.052429 seconds
 Optimal S=64, n=32000, # of comparisons=654999, Time taken=0.069094 seconds
 Optimal S=128, n=32000, # of comparisons=1007191, Time taken=0.087494 seconds
 Optimal S=2, n=64000, # of comparisons=849503, Time taken=0.131314 seconds
 Optimal S=16, n=64000, # of comparisons=903578, Time taken=0.105192 seconds
 Optimal S=64, n=64000, # of comparisons=1359049, Time taken=0.139087 seconds
 Optimal S=128, n=64000, # of comparisons=2059569, Time taken=0.191069 seconds

PS C:\Users\seant> █

timeTaken:.6f} seconds")

Part C (Improved experiment data management)

```
def experiment_fixed_S(sizes, fixed_S, max_val=1000000):  
    global comparisons  
    results = []  
    for n in sizes:  
        arr = generate_random_array(n, max_val)  
        comparisons = 0  
        start = time.time()  
        hybridSort(arr, 0, len(arr)-1, fixed_S)  
        end = time.time()  
        results.append({'n': n, 'S': fixed_S, 'comparisons': comparisons, 'runtime_seconds': end - start})  
    return pd.DataFrame(results)
```

Used **pandas.DataFrame** to store experiment results
(n, S, comparisons, runtime_seconds) for easier plotting and analysis.

Part C (Plotting functionality)

```
# Part (i) Fixed S, varying n
df_i = experiment_fixed_S(sizes, fixed_S)
print("\nPart (i) results:\n", df_i)
df_i.plot(x='n', y='comparisons', marker='o', title=f"Comparisons vs n (S={fixed_S})")
plt.xlabel("Input size n")
plt.ylabel("Comparisons")
plt.grid(True)
plt.show()

# Part (ii) Fixed n, varying S
df_ii = experiment_fixed_n(fixed_n, S_values)
print("\nPart (ii) results:\n", df_ii)
df_ii.plot(x='S', y='comparisons', marker='o', title=f"Comparisons vs S (n={fixed_n})")
plt.xlabel("Threshold S")
plt.ylabel("Comparisons")
plt.grid(True)
plt.show()

# Part (iii) Optimal S
df_iii = experiment_optimal_S(optimal_nsize, optimal_S)
print("\nPart (iii) results:\n", df_iii)
for n in optimal_nsize:
    subset = df_iii[df_iii['n'] == n]
    plt.plot(subset['S'], subset['comparisons'], marker='o', label=f"n={n}")
plt.title("Comparisons vs S for different n")
plt.xlabel("Threshold S")
plt.ylabel("Comparisons")
plt.legend()
plt.grid(True)
plt.show()

for n in optimal_nsize:
    subset = df_iii[df_iii['n'] == n]
    plt.plot(subset['S'], subset['runtime_seconds'], marker='o', label=f"n={n}")
plt.title("Runtime vs S for different n")
plt.xlabel("Threshold S")
plt.ylabel("Runtime (seconds)")
plt.legend()
plt.grid(True)
plt.show()
```

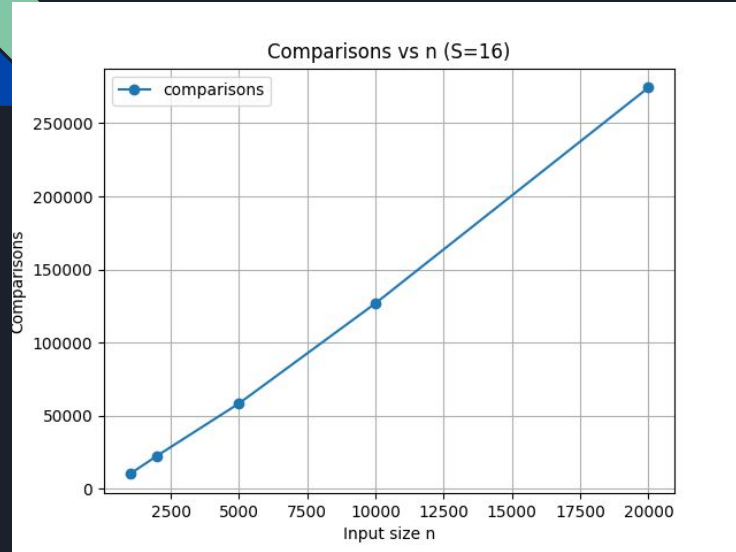
Used matplotlib to visualize experimental results.

Part C(i): Fixed S, plotted comparisons vs n to show how the number of key comparisons changes with input size.

Part C(ii): Fixed n, plotted comparisons vs S to observe how the threshold S affects the number of comparisons.

Part C(iii): Plotted runtime vs S to analyze the effect of different S values on performance.

c(i) Fixed $S = 16$, varying n



Analysis

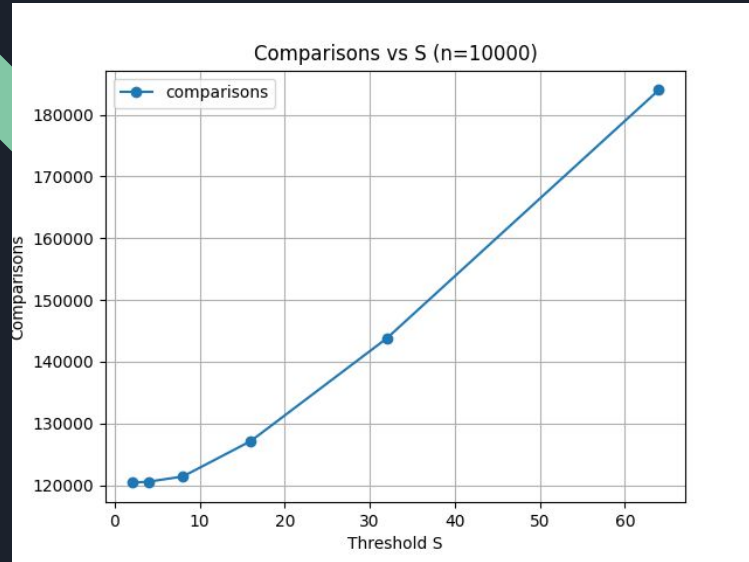
- Key comparisons approximately double when the input size doubles.
- Runtime generally increases proportionally with the number of key comparisons.
- Fixed threshold $S = 16$ keeps the hybrid algorithm efficient for small subarrays.
- Reduces overhead of recursive calls while maintaining merge efficiency.

Part (i) results:

	n	S	comparisons	runtime_seconds
0	1000	16	10245	0.001644
1	2000	16	22737	0.003714
2	5000	16	58502	0.010544
3	10000	16	126940	0.023273
4	20000	16	274383	0.050854

The trend observed in the plot matches the expected $O(n \log n)$ behavior.

c(ii) Fixed $n = 10,000$, varying S



Analysis

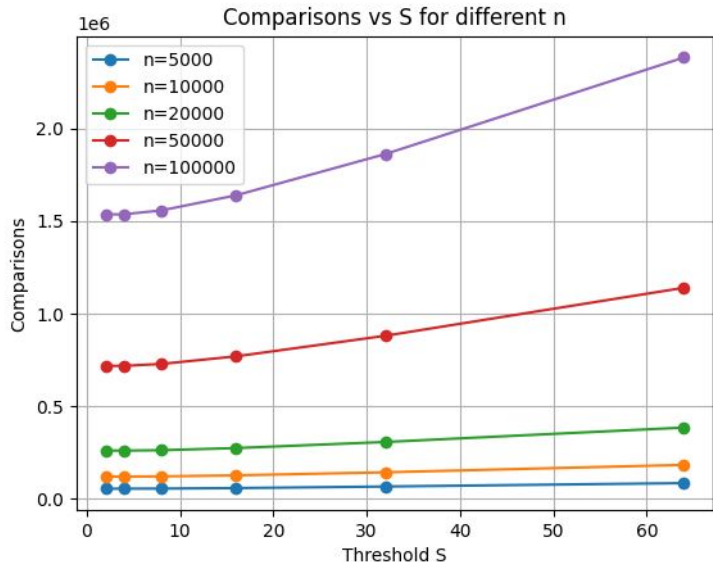
- Larger subarrays handled by insertion sort are less efficient.
- Runtime remains relatively stable for small-to-moderate S due to Python overhead dominating small differences in comparisons.
- Choosing an excessively large S increases comparisons without significantly improving runtime.
- Empirical results show near-monotonic increase: While larger S reduces MergeSort comparisons, it increases Insertion Sort comparisons at a faster rate. The linear growth of Insertion Sort cost $O(nS)$ eventually overwhelms the logarithmic savings from MergeSort $O(n \log(n/S))$.

Part (ii) results:

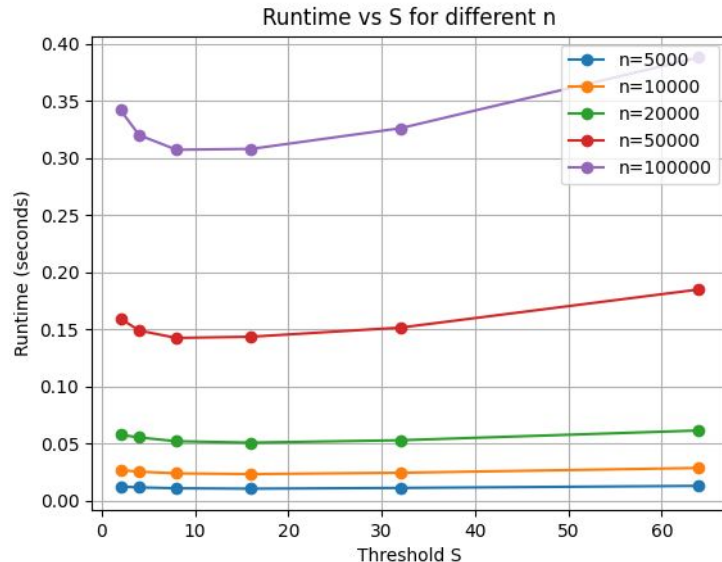
	n	S	comparisons	runtime_seconds
0	10000	2	120436	0.026914
1	10000	4	120602	0.025450
2	10000	8	121433	0.024001
3	10000	16	127174	0.023362
4	10000	32	143805	0.024399
5	10000	64	184000	0.028669

c(iii) Determining the optimal value of S by varying n and S

`optimal_S = [2, 4, 8, 16, 32, 64]`



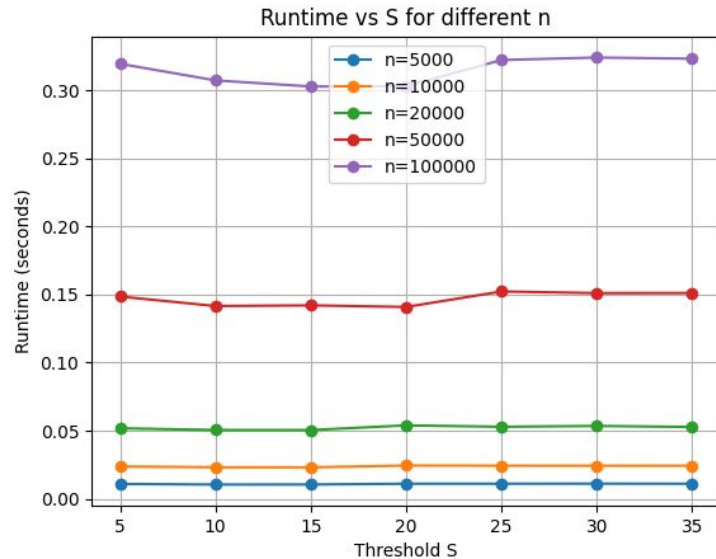
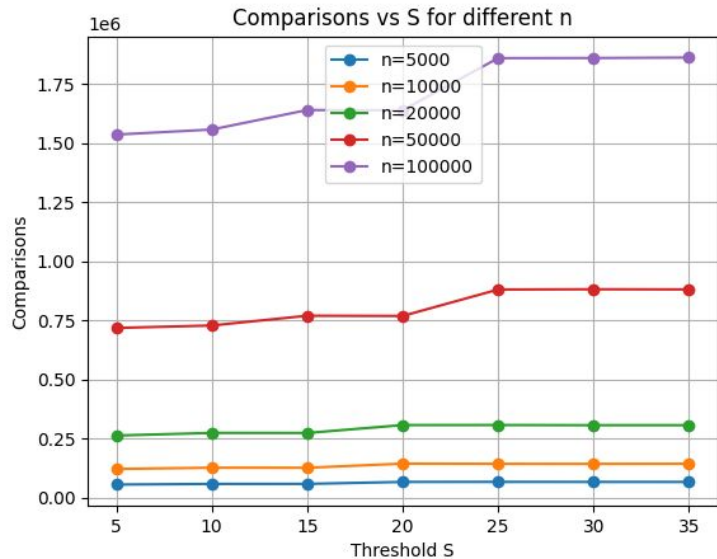
The smaller, the better.
(S=2)



Optimal balance point.
(S=16)

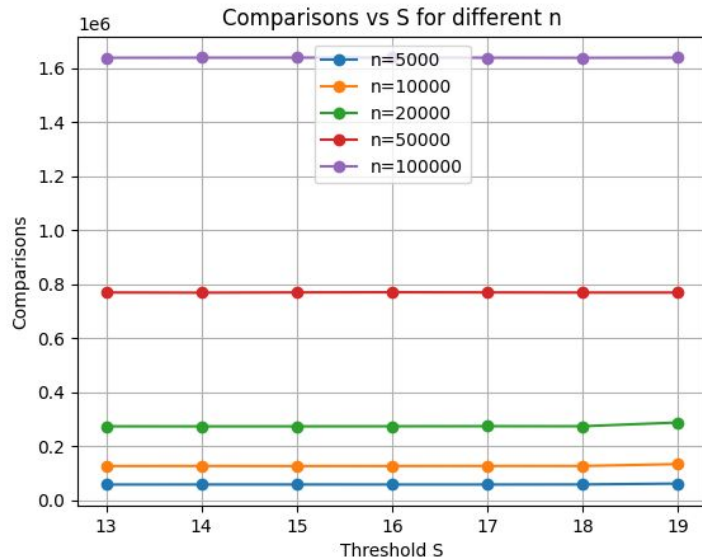
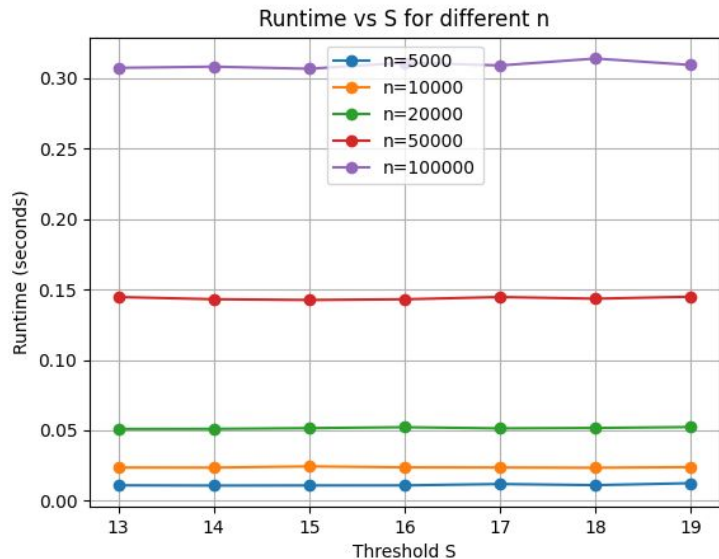
c(iii) Determining the optimal value of S by varying n and S

`optimal_S = [5, 10, 15, 20, 25, 30, 35]`



c(iii) Determining the optimal value of S by varying n and S (fine-grained testing)

optimal_S = [13, 14, 15, 16, 17, 18, 19]



Selected **S=16** based on established computer science standards: power of two (2^4) optimizes for cache lines and memory access patterns.

c(iii) Determining the optimal value of S by varying n and S

Part (iii) results:

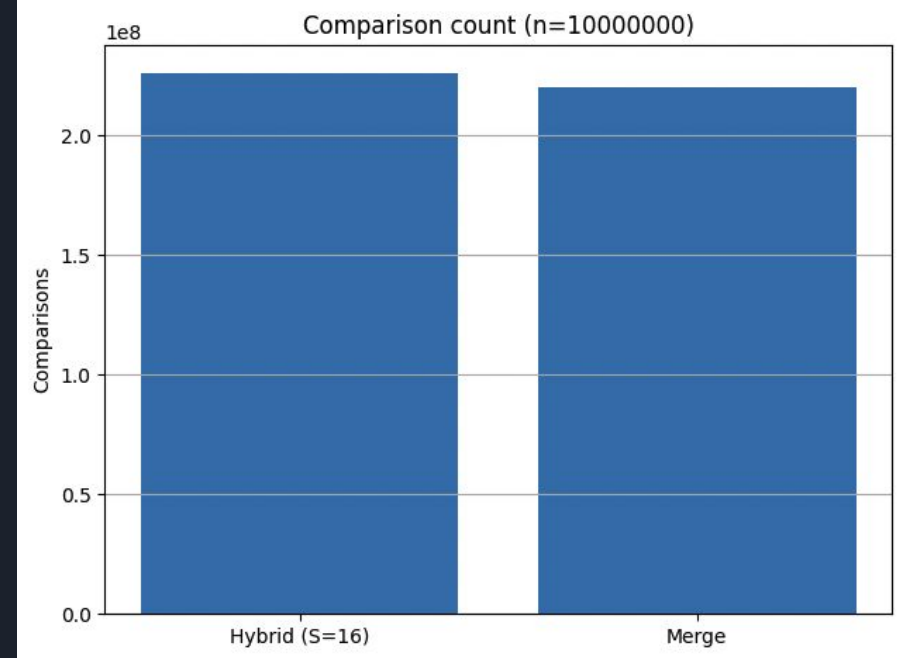
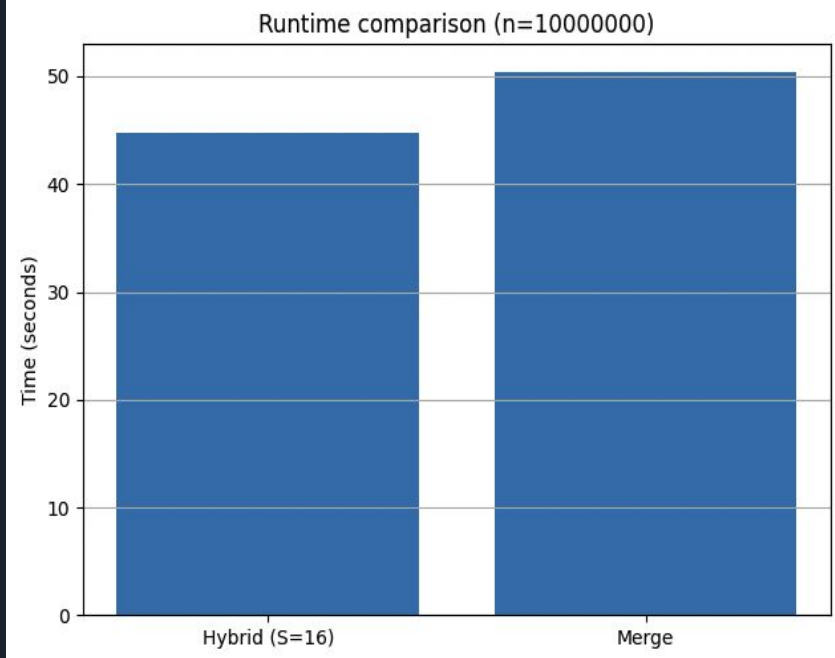
	n	S	comparisons	runtime_seconds
0	5000	2	55185	0.012400
1	5000	4	55215	0.011637
2	5000	8	55729	0.010894
3	5000	16	58382	0.010528
4	5000	32	66839	0.011187
5	5000	64	85270	0.012979
6	10000	2	120453	0.026781
7	10000	4	120605	0.025416
8	10000	8	121480	0.023920
9	10000	16	127190	0.023315
10	10000	32	143509	0.024448
11	10000	64	183654	0.028675
12	20000	2	260961	0.057788
13	20000	4	261000	0.055406
14	20000	8	262883	0.052092
15	20000	16	274501	0.050871
16	20000	32	307392	0.052960
17	20000	64	385116	0.061519
18	50000	2	718391	0.159456
19	50000	4	718340	0.149043
20	50000	8	728827	0.142572
21	50000	16	769851	0.143650
22	50000	32	880887	0.151632
23	50000	64	1140036	0.185075
24	100000	2	1536335	0.342132
25	100000	4	1536987	0.320255
26	100000	8	1557904	0.307448
27	100000	16	1639308	0.308214
28	100000	32	1862224	0.326303
29	100000	64	2383531	0.388294

Analysis

- Too small S increases the number of insertion sort recursive calls.
- Too large S increases the number of comparisons.
- Hybrid algorithm balances recursion overhead and insertion sort efficiency at an intermediate S.

S=16 represents the optimal trade-off between theoretical comparison efficiency and practical computational overhead, as it minimizes the combined cost of comparisons and recursive operations while maintaining good cache performance.

(d) Hybrid sort vs Merge sort



Hybrid sort ran faster but performed slightly more key comparisons than pure Merge sort







